



Università
di **Genova**

DIBRIS: Robotics Engineering

AIRO2 PRESENTATION

ASSIGNMENT D5-V2: AGRICULTURAL ROBOTICS - IRRIGATION PLANNING

Student

Sisani Francesco

Teacher

Mastrogiovanni Fulvio

A.A. 25/26

PROBLEM DEFINITION AND OBJECTIVES

Problem: one robot is deployed in a field divided in crops, as shown in figure 1.

It must be able to bring all the crops to a certain moisture level, assumed as stable, while keeping count of environmental constraints and limited resources

Assumptions:

- robot always starts in start₁;
- locations connected in the four directions;
- moisture level represented in numbers, 50 is the stability threshold

Objective: define solution plans using features from basic PDDL and PDDL+ to:

- use the planner's capabilities;
- explain the course of action that brings to an approximation of realistic scenarios.

Planner used: ENSHP (online and local)

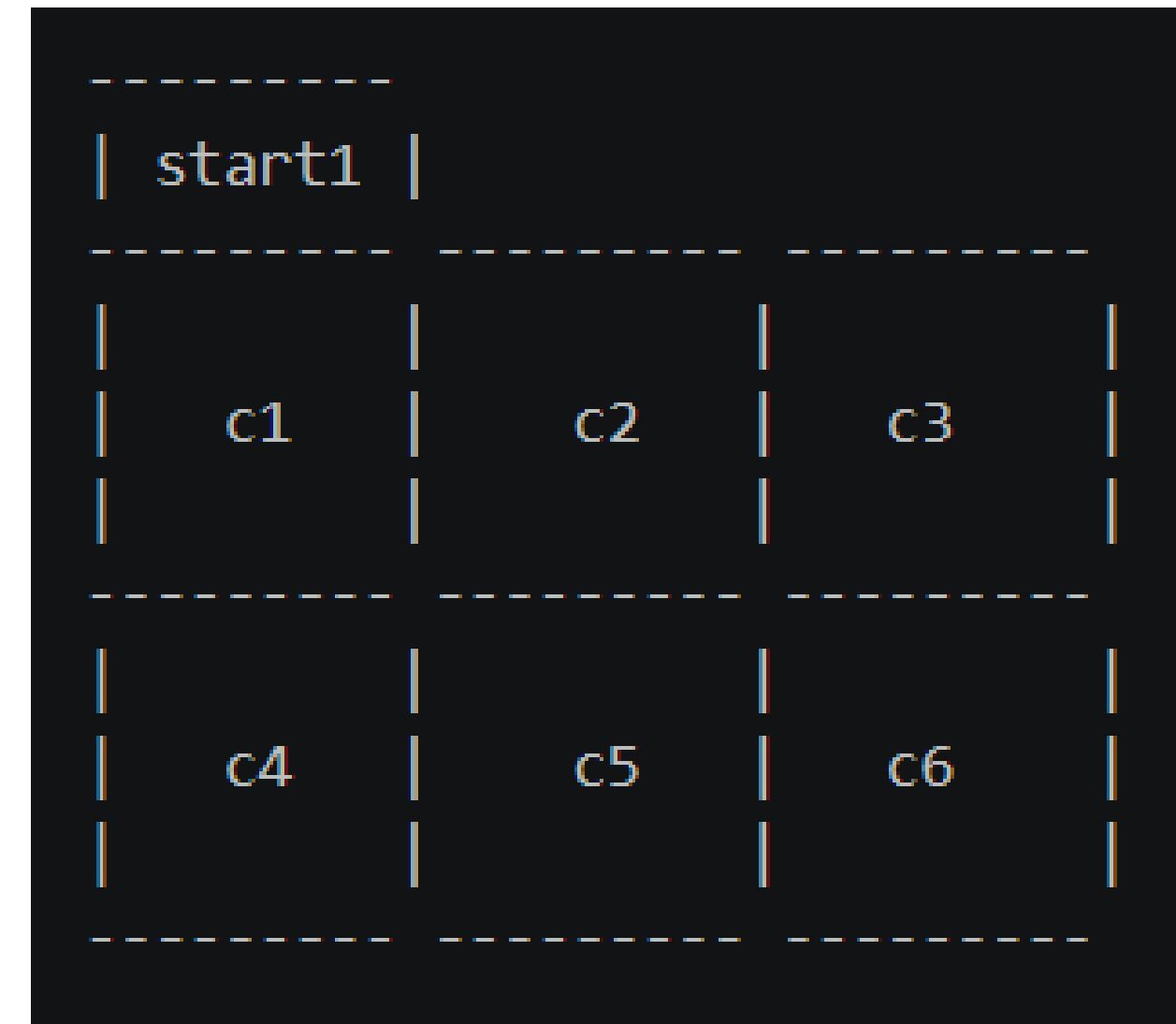


Fig 1: Testing environment

BASIC PDDL DOMAIN

Domain file common to both the abundant and limited resource problem files.

No environmental dynamics involved, used to define the robot's reasoning logic.

Main actions:

1) *Movement*: used to change the robot's position in the field. Allowed between two connected crops.

2) *Check_levels*: checks every crop's moisture level and chooses the one with the lowest level.

3) *Irrigate*: when the robot is located on a prioritized crop, it irrigates that crop by decreasing its water supply by 10 and increasing that crop's moisture level by 10. Numeric fluents avoid binary representation.

4) *Sacrifice*: a crop is flagged as sacrificed when the robot cannot irrigate it due to complete resource depletion.

```
(:action check_levels
  :parameters (?c - crop)
  :precondition (and
    (forall (?other - crop) (>= (moisture_level ?other)
      (moisture_level ?c)))
  )
  :effect (and
    (is-priority ?c)
    (forall (?other - crop)
      (when (not (= ?other ?c)) (not (is-priority ?other)))
    )
  )
)
```

Fig. 2: code structure of priority assignment

Robot logic: Choose priority crop → Reach it → Irrigate it → Repeat

PDDL+ DOMAIN

PDDL+ allows, by introducing continuous time, to add more realism in the current scenario.

Two main modifications emerge when compared to the basic scenario:

- introduced a process which passively drains every crop's moisture level and an event which makes a crop unusable if left alone for too much time;
- modified the logic that regulates movement and irrigation: they are no more instant actions, but processes toggled by actions and events;
- modified the priority logic to make it less computationally heavy in continuous time.

```
(:process evaporation
  :parameters (?c - crop)
  :precondition (and (>= (moisture_level ?c) 10) (not (unusable ?c)))
  :effect (and
    (decrease (moisture_level ?c) (* 1 #t))
  )
)

(:event drought
  :parameters (?c - crop)
  :precondition (and
    (<= (moisture_level ?c) 10)
    (not (unusable ?c))
  )
  :effect (and
    (assign (moisture_level ?c) 0)
    (unusable ?c)
    (increase (num_drought_events) 1)
  )
)
```

Fig.5: environmental conditions implemented

RESULTS - ABUNDANT WATER PROBLEM

Found Plan:

```
0.0: (check_levels c5 robot1)
1.0: (move robot1 start1 c1)
2.0: (move robot1 c1 c4)
3.0: (move robot1 c4 c5)
4.0: (irrigate robot1 c5)
5.0: (irrigate robot1 c5)
6.0: (check_levels c1 robot1)
7.0: (move robot1 c5 c2)
8.0: (move robot1 c2 c1)
9.0: (irrigate robot1 c1)
10.0: (irrigate robot1 c1)
11.0: (irrigate robot1 c1)
12.0: (irrigate robot1 c1)
13.0: (check_levels c5 robot1)
14.0: (move robot1 c1 c2)
15.0: (move robot1 c2 c5)
16.0: (irrigate robot1 c5)
17.0: (irrigate robot1 c5)
18.0: (check_levels c3 robot1)
19.0: (move robot1 c5 c2)
20.0: (move robot1 c2 c3)
21.0: (irrigate robot1 c3)
22.0: (check_levels c5 robot1)
23.0: (move robot1 c3 c2)
24.0: (move robot1 c2 c5)
25.0: (irrigate robot1 c5)
```


RESULTS - LIMITED WATER PROBLEM

Found Plan:

```
0.0: (check_levels c5 robot1)
1.0: (move robot1 start1 c1)
2.0: (move robot1 c1 c4)
3.0: (move robot1 c4 c5)
4.0: (irrigate robot1 c5)
5.0: (irrigate robot1 c5)
6.0: (irrigate robot1 c5)
7.0: (check_levels c1 robot1)
8.0: (move robot1 c5 c2)
9.0: (move robot1 c2 c1)
10.0: (irrigate robot1 c1)
11.0: (irrigate robot1 c1)
12.0: (check_levels c5 robot1)
13.0: (move robot1 c1 c2)
14.0: (move robot1 c2 c5)
15.0: (irrigate robot1 c5)
16.0: (irrigate robot1 c5)
17.0: (check_levels c1 robot1)
18.0: (move robot1 c5 c2)
19.0: (move robot1 c2 c1)
20.0: (irrigate robot1 c1)
21.0: (sacrifice c3 robot1)
22.0: (sacrifice c1 robot1)
```

RESULTS - PDDL+

Found Plan:

```
0: (choose_target robot c5)
0: (start_move robot start1 c1)
0: -----waiting----- [1.0]
1.0: (start_move robot c1 c4)
1.0: -----waiting----- [2.0]
2.0: (start_move robot c4 c5)
2.0: -----waiting----- [4.0]
4.0: (start_irrigation robot c5)
4.0: -----waiting----- [8.0]
8.0: (stop_irrigation robot c5)
8.0: (choose_target robot c1)
8.0: (start_move robot c5 c4)
8.0: -----waiting----- [9.0]
9.0: (start_move robot c4 c1)
9.0: -----waiting----- [10.0]
10.0: (start_irrigation robot c1)
10.0: -----waiting----- [16.0]
16.0: (stop_irrigation robot c1)
16.0: (choose_target robot c3)
16.0: (start_move robot c1 c2)
16.0: -----waiting----- [17.0]
```

```
17.0: -----waiting----- [18.0]
18.0: (start_irrigation robot c3)
18.0: -----waiting----- [22.0]
22.0: (stop_irrigation robot c3)
22.0: (choose_target robot c2)
22.0: (start_move robot c3 c2)
22.0: -----waiting----- [23.0]
23.0: (start_irrigation robot c2)
23.0: -----waiting----- [25.0]
25.0: (stop_irrigation robot c2)
25.0: (choose_target robot c5)
25.0: (start_move robot c2 c5)
25.0: -----waiting----- [26.0]
26.0: (start_irrigation robot c5)
26.0: -----waiting----- [28.0]
28.0: (stop_irrigation robot c5)
28.0: (choose_target robot c6)
28.0: (start_move robot c5 c6)
28.0: -----waiting----- [29.0]
29.0: (start_irrigation robot c6)
29.0: -----waiting----- [30.0]
30.0: (stop_irrigation robot c6)
```

CONCLUSIONS

Main challenges:

- Implementing the priority logic and solving related bottlenecks in both scenarios
- Conversion of movement and irrigation actions in continuous time
- Ordering the actions in both scenarios to make the course of action logically sound

Possible differences from current implementation:

- Priority logic in basic problems is fired after every irrigation step, rather than after a complete irrigation.
Possible to modify it since no environmental dynamics are involved.
- In PDDL+, movement is regulated by a action → process → event logic, while irrigation has a action → process → action logic.

Limitations:

- All problems' solvability is hard-constrained by the water supply's quantity